

Inyección SQL (SQLi)

Objetivos de aprendizaje

Reconstruir la consulta probable, identificar el contexto de inserción, construir un oráculo mínimo, distinguir señales de falsos positivos y adaptar sintaxis solo después de obtener evidencia sobre el motor.

Prerrequisitos

Flujo source-transformación-sink, sintaxis SQL básica, parámetros preparados, HTTP y método de hipótesis.

Fundamentos técnicos

Existe SQLi cuando datos controlados alteran la estructura de una sentencia SQL. La causa no es “aceptar comillas”, sino mezclar estructura y valores en el mismo texto antes de que lo analice el motor. Un valor parametrizado sigue siendo dato aunque contenga SQL; un identificador dinámico, como una columna de orden, requiere seleccionar entre expresiones permitidas por la aplicación.

Modelo mental

entrada -> decode/coerción -> concatenación o binding -> parser SQL
-> plan/ejecución -> filas, error, booleano, tiempo u OOB

Antes de escribir un payload responde: ¿el valor está dentro de un literal, es una expresión numérica, ocupa una lista, un ORDER BY o ni siquiera llega a SQL?

Superficie de ataque

Buscadores, filtros, login, IDs, cookies de preferencias, cabeceras usadas en auditoría y cuerpos JSON pueden alimentar consultas. El nombre del parámetro no importa; importa el flujo hasta el motor.

Cómo identificarla

- Parejas verdadera/falsa producen diferencias reproducibles dependientes de la condición.
- Un error atribuido al motor revela contexto, pero por sí solo es un indicio.
- Una demora confirma un canal solo si escala con el valor solicitado, se repite y se separa de controles.
- UNION solo es útil si la consulta devuelve filas, coincide el número de columnas y hay tipos compatibles.

Preguntas que debo hacerme

1. ¿Qué consulta explica la respuesta normal?
2. ¿El punto de inserción es textual, numérico o estructural?
3. ¿Qué sufijo queda después de mi entrada?
4. ¿Qué canal observable tengo?
5. ¿Qué comportamiento alternativo produciría la misma señal?

Prueba mínima

Una comilla es una sonda de sintaxis, no una confirmación. La prueba mínima preferida es una pareja válida para el contexto que cambie solo una condición, acompañada de un valor inexistente y repetición.

Construcción progresiva del payload

Escenario base:

```
SELECT id, name, price FROM products WHERE category = '<entrada>' AND visible = 1
```

1. Entrada válida: gifts.
2. Reconstrucción: el valor está dentro de comillas y queda el sufijo ' AND visible = 1.

3. Primitiva: variar una condición sin extraer datos.
4. Variante verdadera: cerrar el literal, añadir una condición verdadera y neutralizar el sufijo según el motor confirmado.
5. Variante falsa: misma estructura, una condición falsa.
6. Solo si existe un canal de filas se estudia UNION; primero número de columnas, luego compatibilidad de tipos y columna reflejada.

No se encadenan detección, identificación de motor y extracción en una única cadena: cada paso debe resolver una incertidumbre.

Anatomía de los payloads

- Contexto de entrada: query category.
- Sintaxis original: `WHERE category = '<entrada>' AND visible = 1.`
- Entrada controlada: interior de un literal textual.
- Transformaciones conocidas: URL decode una vez, sin coerción.
- Parser final: PostgreSQL confirmado en el laboratorio por error de versión controlado.
- Sink: ejecución de `SELECT`.
- Primitiva: oráculo booleano.
- Payload mínimo: `gifts' AND '1'='1' --` , emparejado con `gifts' AND '1'='2' --` .
- Significado de cada componente: `gifts'` cierra el valor; `AND` añade condición; la comparación cambia verdad/falsedad; `--` neutraliza el sufijo en este contexto.
- Resultado esperado: la pareja produce presencia/ausencia estable del producto.
- Control negativo: categoría inexistente y petición original.
- Restricción observada: el sufijo provoca error si no se neutraliza.
- Por qué falla la variante básica: `' OR '1'='1` deja comillas y `AND visible=1` en una estructura distinta de la asumida.
- Hipótesis de adaptación: reconstruir y neutralizar exactamente el sufijo.
- Payload adaptado: la pareja anterior.
- Por qué debería funcionar: ambas consultas resultantes son válidas y difieren solo en una condición.
- Evidencia: diferencia reproducible en cuerpo y número de filas.

Cuándo no funcionaría: binding parametrizado, contexto numérico, comentarios distintos o filtro semántico.

Variaciones según el contexto

Contexto probable	Reconstrucción	Primera pareja útil
Numérico	<code>WHERE id = <entrada></code>	<code>12 AND 1=1 / 12 AND 1=2</code>
Texto	<code>WHERE name = '<entrada>'</code>	cerrar y construir dos comparaciones válidas
ORDER BY	<code>ORDER BY <entrada></code>	valores permitidos que revelen orden; no tratar como literal
LIKE	<code>LIKE '%<entrada>'</code>	considerar comodines y ambos delimitadores

Las funciones de versión, concatenación, demora, catálogos y comentarios cambian entre motores. Se consultan después de identificar el motor; no se usan como detector universal.

Filtros y bypasses

Un rechazo puede venir de conversión de tipo, WAF, parser HTTP o motor. Compara forma enviada, valor recibido y consulta final. Sustituir espacios, mayúsculas o encoding solo está justificado si una prueba localiza la comparación que se intenta eludir.

Evidencias de confirmación

La mejor evidencia es causal: dos consultas que solo difieren en una condición producen resultados opuestos de forma repetible. Un 500, una página vacía o una demora aislada no confirman SQLi.

Escalado de impacto

Tras confirmar la primitiva: identificar motor y versión solo si aporta sintaxis; determinar canal; verificar columnas y tipos si procede; demostrar acceso mínimo a un dato no sensible del laboratorio; documentar privilegios de la cuenta. Automatizar no sustituye la confirmación manual ni autoriza un volcado indiscriminado.

Errores frecuentes

- Usar payload textual en contexto numérico.
- Asumir que `information_schema`, comentarios o funciones son iguales en todos los motores.
- Interpretar 500 como confirmación.
- Usar UNION sin conocer columnas, tipos o canal de renderizado.
- Cambiar condición, comentario y encoding simultáneamente.

Diagnóstico de payloads fallidos

Síntoma	Hipótesis	Experimento discriminatorio
' produce 500, pero cualquier símbolo también	conversión o error genérico	comparar caracteres inocuos y revisar fase del error
1 AND 1=1 y 1 AND 1=2 son iguales	binding, coerción o canal sin filas	valor inexistente, log de consulta y otro canal controlado
ORDER BY 4 falla	índice fuera de rango o lógica de aplicación	comparar 1..4 y conservar cuerpos completos
UNION siempre falla	columnas/tipos/contexto/motor	reducir a NULL, reconstruir sufijo y consultar sintaxis confirmada
demora variable no proporcional	carga, lock o reintento	alternar controles y dos duraciones en varias muestras
error nombra otro parser	la entrada no llegó a SQL	corregir JSON, tipo o transporte antes de adaptar SQL

Mitigaciones

Consultas preparadas con binding para valores, mapas cerrados para identificadores dinámicos, privilegio mínimo, errores externos no verbosos y pruebas sobre la frontera exacta. Escapar cadenas es frágil y dependiente del motor.

Relación con pentesting y certificaciones

Se evalúa la capacidad de justificar la consulta reconstruida, el oráculo y los controles, no la memoria de cadenas ni la cantidad de datos extraídos.

Caso guiado

Caso A — Básico: filtro de precio numérico

GET /search?max=50 devuelve 12 productos; max=50 AND 1=1 devuelve 12 y max=50 AND 1=2 devuelve 0.

Observación: una pareja lógica cambia filas. Qué sé realmente: el valor acepta sintaxis compatible con una expresión booleana. Hipótesis: SQL concatenado o evaluador previo. Experimento discriminatorio: valor inexistente, expresión propia del SQL estándar y log de consulta del laboratorio. Resultado esperado: SQL concatenado conserva la expresión en WHERE price <= ...; un evaluador enviaría un número calculado. Resultado obtenido: el log muestra price <= 50 AND 1=2. Conclusión: SQLi numérica con canal booleano. Siguiente paso: identificar motor antes de usar funciones específicas.

Caso de adaptación

Caso B — La comilla no funciona

En /item?id=7, ' responde 400 integer required, pero 7-1 devuelve el artículo 6.

Observación: la comilla muere antes del sink; aritmética cambia el objeto. Qué sé: existe coerción parcial o contexto numérico. Hipótesis: H1, validación permite expresiones y SQL las evalúa; H2, la aplicación evalúa y parametriza. Experimento: pareja 7 AND 1=1 / 7 AND 1=2 y trazas del laboratorio. Predicción: H1 produce diferencia en filas; H2 rechazará AND o enviará un entero. Resultado: diferencia estable y SQL final visible. Conclusión: la adaptación correcta elimina comillas; no busca codificarlas. Siguiente paso: documentar el oráculo.

Caso C — Transferencia: preferencia de orden

Una cookie sort=price cambia el orden del catálogo. No se nombra la vulnerabilidad. El alumno debe mapear cookie -> selección de columna -> ORDER BY. Un intento con comillas no cambia nada. Cambiar entre price y name sí altera el orden; un valor fuera de catálogo produce un error de columna del motor.

Resolución: la entrada no ocupa un literal sino una posición estructural. El experimento correcto compara columnas existentes y un valor inválido; la defensa es un mapa cerrado de nombres públicos a expresiones SQL, no intentar parametrizar el identificador como valor.

Caso D — Falso positivo: el 500 de la comilla

Una API declara id como UUID. Cualquier valor con longitud distinta de 36, incluida ', produce el mismo 500 por una excepción del validador.

Observación: la comilla causa error. Qué sé: solo que el input es inválido. Hipótesis: SQLi o validación defectuosa. Experimento: varios UUID inválidos de igual longitud, un UUID válido inexistente y logs de capa. Resultado: todos los inválidos fallan antes del repositorio; el válido inexistente produce 404. Conclusión: no hay evidencia de SQLi. Siguiente paso: reportar manejo de errores solo si entra en alcance.

Ejercicios

1. Reconstruye SELECT ... WHERE code='<entrada>' AND tenant=? y diseña una pareja que no altere el tenant.
2. Una demora de 3 s aparece en 2 de 10 payloads y 3 de 10 controles: decide qué medir ahora.
3. Explica por qué UNION SELECT NULL,NULL no prueba nada si el resultado no se renderiza.
4. Diseña una prueba que distinga coerción a entero de SQL numérico concatenado.

Resumen

SQLi se demuestra reconstruyendo contexto y obteniendo un oráculo causal. Los payloads se derivan de la consulta y del motor confirmado; no se eligen de una lista.

Chuleta operativa

1. Entrada válida y consulta probable.
2. Contexto textual, numérico o estructural.

3. Sufijo restante.
4. Pareja verdadera/falsa y control inexistente.
5. Canal reproducible.
6. Motor antes de sintaxis específica.
7. Impacto mínimo y mitigación causal.

Referencias

- [PortSwigger: SQL injection](#)
- [PortSwigger: sintaxis específica por motor](#)
- [OWASP: SQL Injection Prevention](#)
- [Concepto interno \(sql_i\)](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Práctica: [cuaderno](#).